

**Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

**Лабораторна робота №8
з дисципліни
«Основи розробки програмного забезпечення мовою
програмування Python»
на тему
“Класи та об’єкти: основи об’єктно-орієнтованого програмування в
Python”**

Виконав:
студент групи ІМ-42
Жила Іван Дмитрович
номер варіанту: 8

Перевірив:
Фещенко К. Ю

Мета роботи

Ознайомитися з основними поняттями об'єктно-орієнтованого програмування (ООП) у мові Python; набути практичних навичок створення класів і об'єктів, використання конструктора класу, атрибутів та методів; навчитися моделювати предметну область із застосуванням класів.

Короткі теоретичні відомості:

1. Поняття об'єктно-орієнтованого програмування (ООП)

Об'єктно-орієнтоване програмування — це підхід до розроблення програмного забезпечення, за якого програма розглядається як сукупність об'єктів, що взаємодіють між собою. Кожен об'єкт поєднує дані та методи їх обробки. Основними поняттями ООП є клас, об'єкт, атрибут, метод та конструктор.

2. Клас та об'єкт у мові Python

- Клас — це шаблон (модель), що визначає структуру та поведінку об'єктів певного типу. Він описує, які дані зберігає об'єкт та які дії з ними можна виконувати. Оголошення класу в Python здійснюється за допомогою ключового слова `class`.

- Об'єкт — це конкретний екземпляр класу, створений на його основі. Один клас може мати необмежену кількість об'єктів, кожен з яких має власні значення атрибутів.

3. Атрибути та методи класу

- Атрибути класу — це змінні, що описують властивості об'єкта та зберігають дані, які характеризують його стан.

- Методи класу — це функції, визначені всередині класу, які описують дії, що може виконувати об'єкт. Методи мають доступ до атрибутів об'єкта.

Для звернення до атрибутів і методів використовується оператор крапки (.).

4. Конструктор класу та ключове слово self

- Конструктор — це спеціальний метод класу, який автоматично викликається під час створення об'єкта. У Python конструктором є метод `__init__`. Його призначення полягає в ініціалізації атрибутів об'єкта та заданні його початкового стану.

- Ключове слово `self` — це обов'язковий перший параметр у визначенні методів класу, який є посиланням на поточний об'єкт. Воно дозволяє отримувати доступ до атрибутів об'єкта та викликати інші методи цього ж об'єкта.

5. Інкапсуляція в Python

- Інкапсуляція — це принцип ООП, що полягає в об'єднанні даних та методів їх обробки в одному класі. Вона дозволяє приховати внутрішню реалізацію об'єкта та надавати доступ до даних через методи.

- У мові Python інкапсуляція реалізується на рівні домовленостей (особливостей іменування атрибутів і методів), що забезпечує логічну ізоляцію даних.

Загальне завдання:

Написати програму, яка:

- вводить початкові дані;
- формує результат з застосуванням концепції ООП;
- виводить результат роботи.

Індивідуальне завдання - Написати клас, який обчислює площу та периметр прямокутника. (8 варіант)

Текст програми:

```
# Лабораторна 8

# Тема: Класи та об'єкти: основи об'єктно-орієнтованого програмування в Python.
```

```
# Загальне завдання: Написати програму, яка вводить початкові дані,
формує результат з застосуванням
# концепції ООП та виводить результат роботи.

# Варіант 8: Написати клас, який обчислює площу та периметр
прямокутника.

# Оголошення класу Student для демонстрації загального завдання
class Student:
    # Конструктор класу (метод ініціалізації об'єкта)
    def __init__(self, first_name, last_name, course):
        # Атрибути класу [cite: 301]
        self.first_name = first_name # ім'я студента
        self.last_name = last_name # прізвище студента
        self.course = course # курс студента

    # Метод для виведення інформації про студента
    def show_info(self):
        print(f"Студент: {self.first_name} {self.last_name} | Курс:
{self.course}")

# Оголошення класу Rectangle для індивідуального завдання
class Rectangle:
    # Конструктор класу для задання початкового стану прямокутника [
    def __init__(self, width, height):
        self.width = width # ширина прямокутника
        self.height = height # висота прямокутника

    # Метод для обчислення площі прямокутника
    def calculate_area(self):
        return self.width * self.height

    # Метод для обчислення периметра прямокутника
    def calculate_perimeter(self):
        return 2 * (self.width + self.height)

    # Метод класу для виведення результатів роботи об'єкта
    def show_results(self):
        print(f"Характеристики прямокутника [{self.width} x {self.height}]:")
        print(f"- Площа: {self.calculate_area()}")
        print(f"- Периметр: {self.calculate_perimeter()}")

# Функція для виконання загального завдання (введення, ООП обробка,
виведення)
def process_general_task():
    print("\nЗагальне завдання: Введення даних студента")

    # Введення початкових даних користувачем
```

```

f_name = input("Введіть ім'я студента: ")
l_name = input("Введіть прізвище студента: ")

try:
    c_num = int(input("Введіть курс (ціле число): "))
    # Формування результату із застосуванням концепції ООП
    student = Student(f_name, l_name, c_num)
    # Виведення результату роботи об'єкта
    student.show_info()
except ValueError:
    print("Помилка: курс повинен бути цілим числом.")
finally:
    print("Блок загального завдання завершив роботу.")

# Функція для виконання індивідуального завдання
def process_individual_task():
    print("\nІндивідуальне завдання (Варіант 8): Клас Rectangle")

    try:
        # Введення початкових даних для прямокутника
        w = float(input("Введіть ширину прямокутника: "))
        h = float(input("Введіть висоту прямокутника: "))

        # Створення об'єкта класу Rectangle (екземпляра класу)
        rect = Rectangle(w, h)
        # Виклик методу для обчислення та виведення результатів
        rect.show_results()
    except ValueError:
        print("Помилка: сторони прямокутника мають бути числовими значеннями.")
    finally:
        print("Обробка індивідуального завдання завершена.")

def main():
    # Виклик та виконання загального завдання
    process_general_task()

    # Виклик та виконання індивідуального завдання відповідно до варіанта
    process_individual_task()

if __name__ == "__main__":
    main()

```

Скріншоти:

Загальне завдання: Введення даних студента

Введіть ім'я студента: *Іван*

Введіть прізвище студента: *Жила*

Введіть курс (ціле число): *2*

Студент: Іван Жила | Курс: 2

Блок загального завдання завершив роботу.

Індивідуальне завдання (Варіант 8): Клас Rectangle

Введіть ширину прямокутника: *5*

Введіть висоту прямокутника: *3*

Характеристики прямокутника [5.0 x 3.0]:

- Площа: 15.0

- Периметр: 16.0

Обробка індивідуального завдання завершена.

Висновок:

Під час виконання лабораторної роботи я ознайомився з основними поняттями об'єктно-орієнтованого програмування (ООП) у мові Python. Набув практичних навичок створення класів та об'єктів, використання конструктора `__init__`, а також оголошення атрибутів і методів для моделювання предметної області.

У ході роботи я успішно реалізував індивідуальне завдання (Варіант 8), створивши клас `Rectangle` для обчислення площі та периметра прямокутника на основі введених користувачем даних. Виконання цієї роботи допомогло мені зрозуміти переваги ООП підходу, який дозволяє ефективно структурувати програмний код, об'єднуючи дані та логіку їхньої обробки в межах одного класу.